



香港中文大學(深圳)

The Chinese University of Hong Kong, Shenzhen

Operating Systems Lecture 24

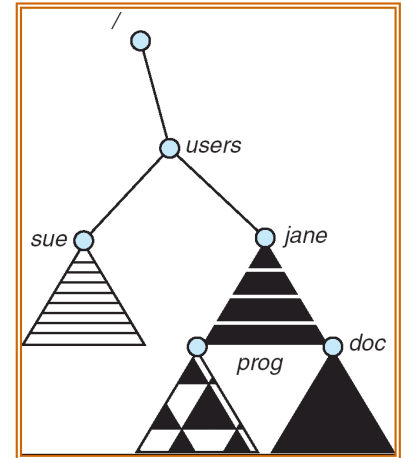
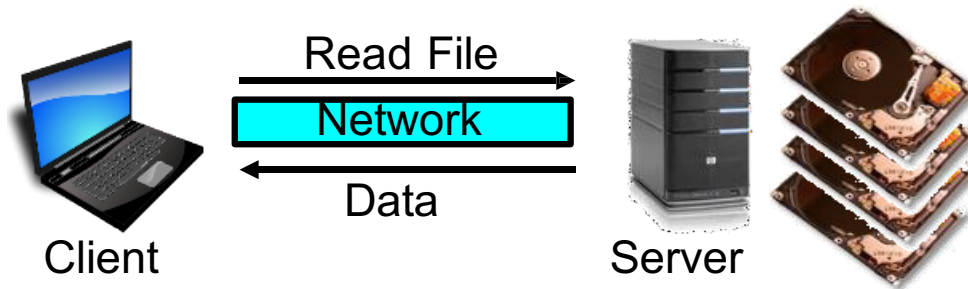
File system 5: data processing systems

**Prof. Yunming XIAO
School of Data Science**

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

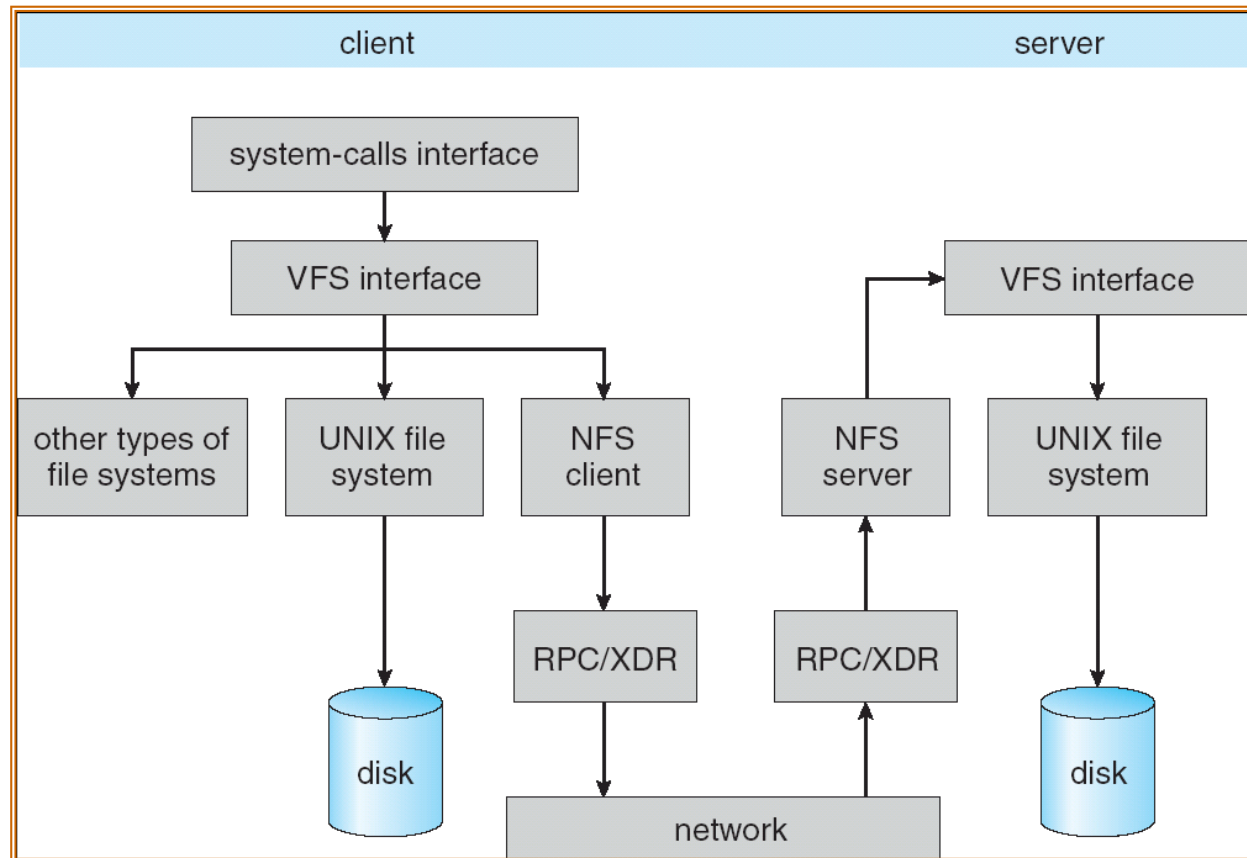
Recall

Recall: distributed file systems



- Transparent access to files stored on a remote disk
- *Mount* remote files into your local file system
 - Directory in local file system refers to remote files
 - e.g., `/users/jane/prog/foo.c` on laptop actually refers to `/prog/foo.c` on `fs.cs.berkeley.edu`

Recall: NFS architecture



Recall: stateless protocols

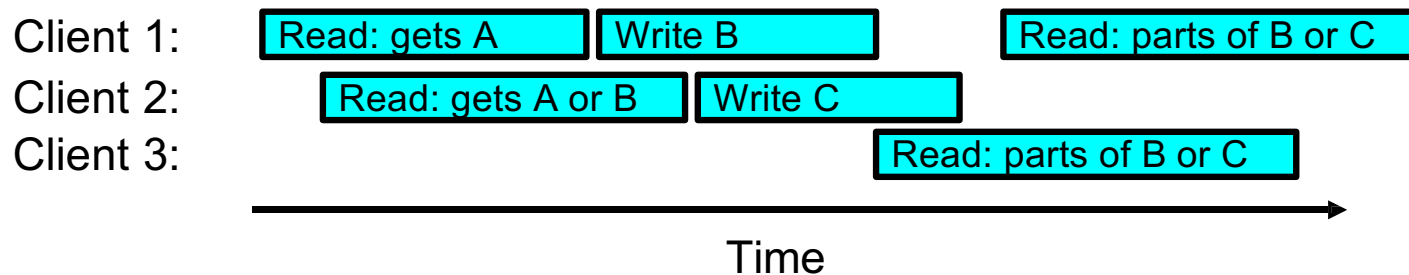
- **Stateless Protocol:** A protocol in which all information required to service a request is included with the request
- **Idempotent Operations** – repeating an operation multiple times is same as executing it just once (e.g., storing to a mem addr.)
- E.g.
“ReadAt(file_id, offset, length)” instead of
“Read(fd, length)”

Recall: NFS cache consistency

- NFS protocol: weak consistency
 - Client polls server periodically to check for changes
 - Polls server if data hasn't been checked in last 3-30 seconds (exact timeout is tunable parameter).
 - Thus, when file is changed on one client, server is notified, but other clients use old version of file until timeout.
- What if multiple clients write to same file?
 - In NFS, can get either version (or parts of both)
 - Completely arbitrary!

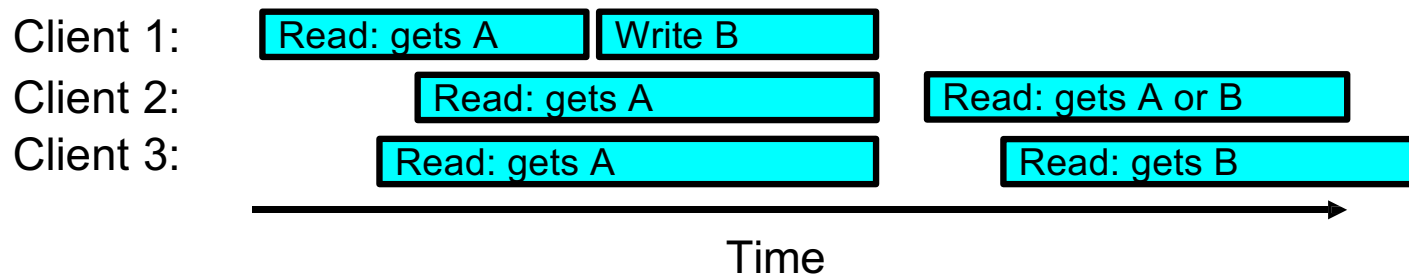
Sequential ordering constraints

- What sort of cache coherence might we expect?
 - i.e. what if one CPU changes file, and before it's done, another CPU reads file?
- Example: Start with file contents = "A"



Sequential ordering constraints

- What sort of cache coherence might we expect?
 - i.e. what if one CPU changes file, and before it's done, another CPU reads file?
- Example: Start with file contents = "A"



Sequential ordering constraints

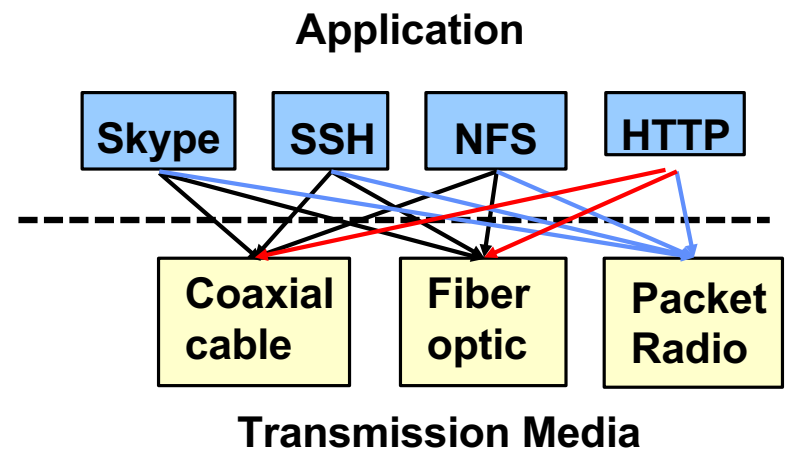
- What would we actually want?
 - Assume we want distributed system to behave exactly the same as if all processes are running on single system
 - If read finishes before write starts, get old copy
 - If read starts after write finishes, get new copy
 - Otherwise, get either new or old copy
- For NFS:
 - If read starts more than 30 seconds after write, get new copy; otherwise, could get partial update

NFS pros and cons

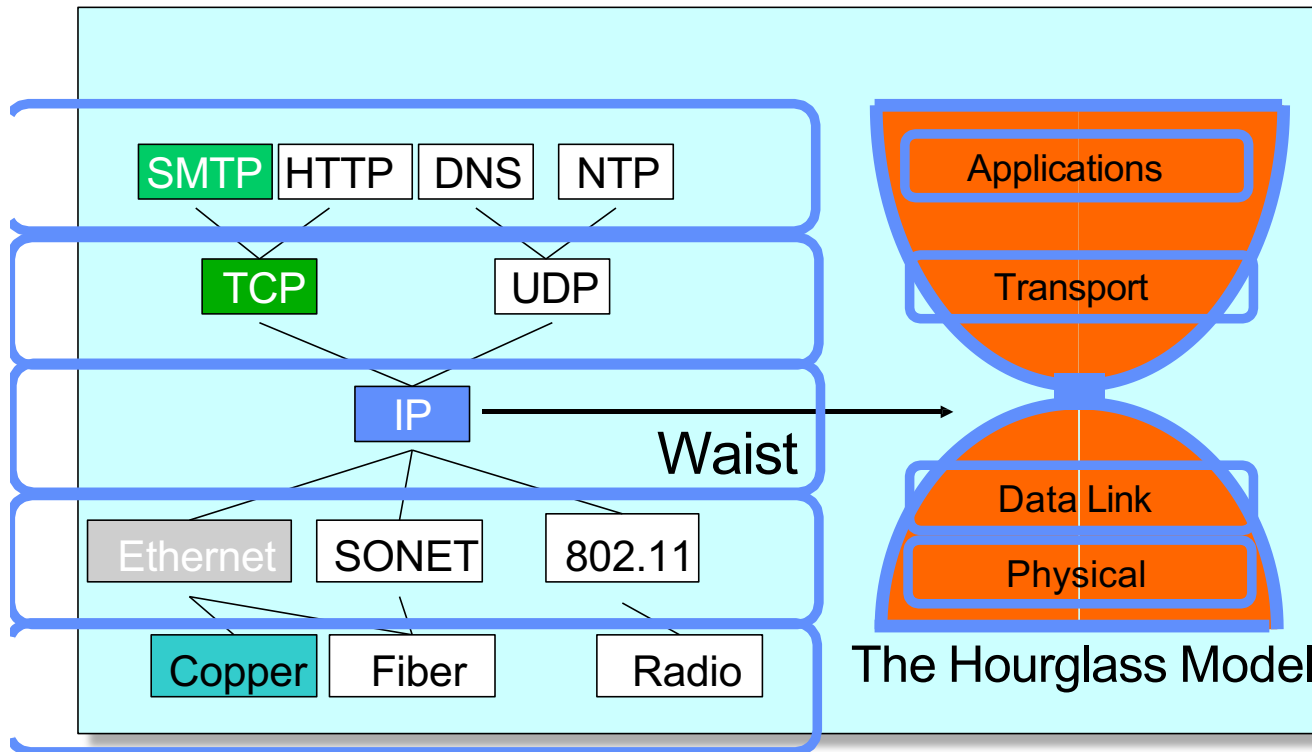
- NFS Pros:
 - Simple, Highly portable
- NFS Cons:
 - Sometimes inconsistent!
 - Doesn't scale to large # clients
 - Must keep checking to see if caches out of date
 - Server becomes bottleneck due to polling traffic

Case study: the Internet

- The Internet is the largest distributed system that exists!
- Many different applications
 - Email, web, P2P, etc.
- Many different operating systems and devices
- Many different network styles and technologies
 - Wireless, wired, optical
- How do we organize this mess
 - Layering & end-to-end principle



The Internet: hourglass model



- “Narrow waist” facilitates *interoperability*
- Layers “abstract” away hardware so that upper layers are agnostic to lower layers
=> Sound familiar?

Implications of hourglass

- Single Internet-layer module (**IP**):
 - Allows arbitrary networks to interoperate
 - Any network technology that supports IP can exchange packets
 - Allows applications to function on all networks
 - Applications that can run on IP can **use any network**
 - Supports simultaneous innovations above and below IP
 - But changing IP itself, i.e., **IPv6**, very involved

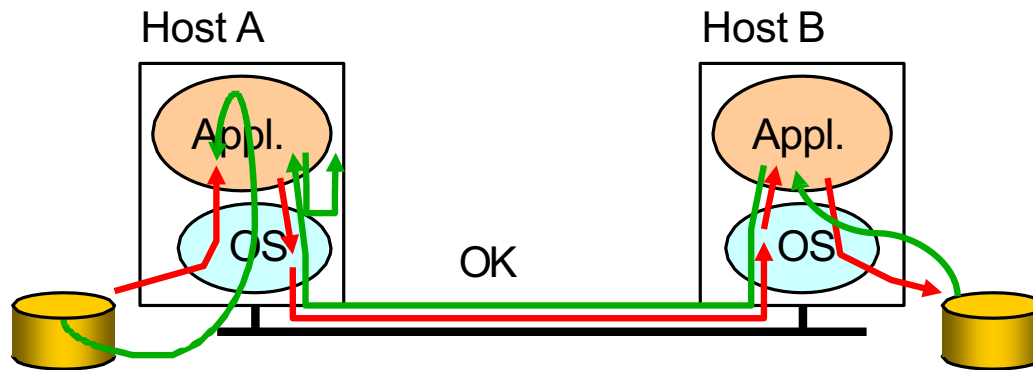
Drawbacks of layering

- Layer N may duplicate layer N-1 functionality
 - E.g., error recovery to retransmit lost data
- Layers may need same information
 - E.g., timestamps, maximum transmission unit size
- Layering can hurt performance
 - E.g., hiding details about what is really going on
- Some layers are not always cleanly separated
 - Inter-layer dependencies for performance reasons
 - Some dependencies in standards (header checksums)

2nd design idea: end-to-end argument

- Hugely influential paper:
 - “End-to-End Arguments in System Design” by Saltzer, Reed, and Clark (‘84)
- “Sacred Text” of the Internet
 - Endless disputes about what it means
 - Everyone cites it as supporting their position
- Simple Message: Some types of network functionality can only be correctly implemented **end-to-end**
 - Reliability, security, etc.
 - Hosts cannot rely on the network help to meet requirement, so must implement it themselves

Example: reliable file transfer



- Solution 1: make each step reliable, and then concatenate them
- Solution 2: end-to-end check and try again if necessary

Discussion

- Solution 1 is incomplete
 - What happens if memory is corrupted? Receiver has to do the check anyway!
- Solution 2 is complete
 - Full functionality can be entirely implemented at application layer with no need for reliability from lower layers
- *Is there any need to implement reliability at lower layers?*
 - Well, it could be more efficient

End-to-end principle

- Implementing complex functionality in the network:
 - Doesn't always reduce host implementation complexity
 - Does increase network complexity
 - Probably imposes delay and overhead on all applications, even if they don't need functionality
- However, implementing in network can enhance performance in some cases
 - – e.g., very lossy link

How to interpret E2E argument?

- **Conservative interpretation:** Don't implement a function at a lower level of the system unless it can be completely implemented at the low level
- **Moderate interpretation:** Think twice before implementing functionality in network
 - If hosts can do the functionality E2E, do it in network only as perf enhancement
 - Don't put a burden on apps that don't need the functionality

Distributed data processing

Topic Roadmap

Distributed File System: NFS

Peer-To-Peer System: The Internet

Distributed Data Processing
(MapReduce and Spark)

Coordination
(Atomic Commit and Consensus)

Case study: distributed data processing



The big data problem

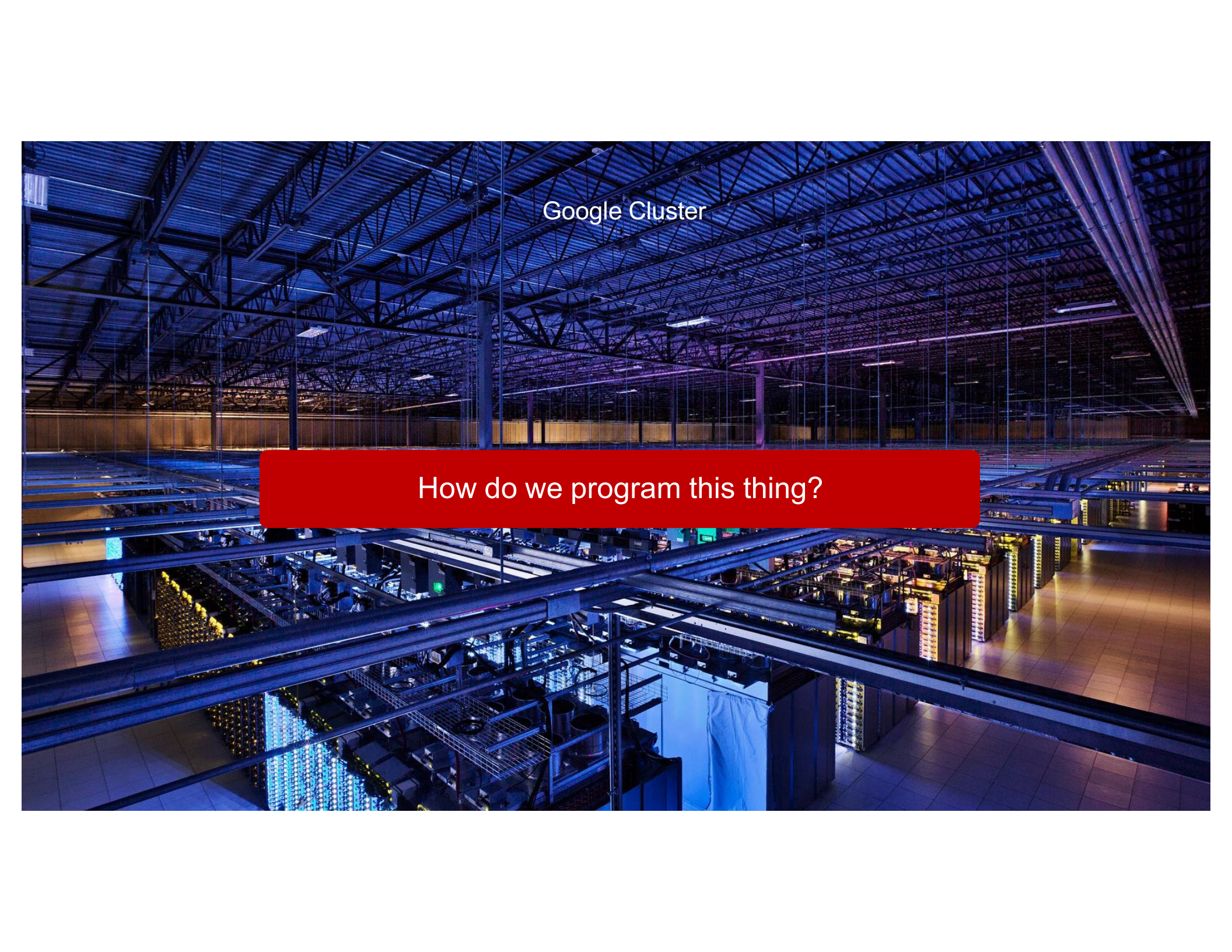
- Data is growing faster than server speeds
- Growing data sources
 - Web, mobile, scientific, ...
- Cheap storage
 - Doubling every ~18 months
- Stalling CPU speeds



Examples

- 1000 genomes project: 200 TB
- Google web index: 100+ PB
- Ingest per day at large Databricks customer: 1 PB
- Cost of 1 TB of disk: \$20
- Time to read 1 TB from disk: 3 hours (at 100 MB/s)

A single machine can't store or process all the data.
Need to **distribute** it over clusters!

A wide-angle photograph of a massive server hall. The room is filled with rows of server racks, some of which are illuminated with blue and yellow lights. The ceiling is high and features a complex network of steel beams and hanging cables. The floor is made of large, light-colored tiles. The overall atmosphere is industrial and high-tech.

Google Cluster

How do we program this thing?

Traditional network programming

- Message-passing between nodes
- **Really hard** to do at scale:
 - How to divide problem across nodes?
 - How to deal with failures?
 - Even worse: stragglers (node is not failed, but slow)

Almost nobody does this for distributed data processing

Distributed data processing frameworks

- Come up with a model for breaking large computations into smaller tasks, then build a framework that distributes those tasks to workers in a cluster
- Framework handles scheduling, fault recovery, etc
- Recent wave popularized with [MapReduce](#) (and open source Hadoop)

MapReduce history

- Developed by Google, paper published in 2004
- Google had lots of raw data:
 - Crawled web pages
 - Server logs
 - Search data
- Wanted to build many apps: web indexing, usage analysis, spam filtering, etc

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

Programs written in this functional style are automatically parallelized and executed on a large cluster of commodity machines. The run-time system takes care of the details of partitioning the input data, scheduling the program's execution across a set of machines, handling ma-

given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computations we were trying to perform but hides the messy details of parallelization, fault-tolerance, data distribution and load balancing in a library. Our abstraction is inspired by the *map* and *reduce* primitives present in Lisp

MapReduce history

- Environment: clusters of cheap commodity machines
 - “Off-the-shelf” machines, i.e. hardware not custom-built for reliability
- Many “one-off” solutions for parallelizing workloads
 - Hard to maintain
 - Hard to get right
 - Time-consuming to implement

MapReduce programming model

- Data type: key-value *records*

- Map function:

$$(K_{in}, V_{in}) \rightarrow \text{list}(K_{inter}, V_{inter})$$

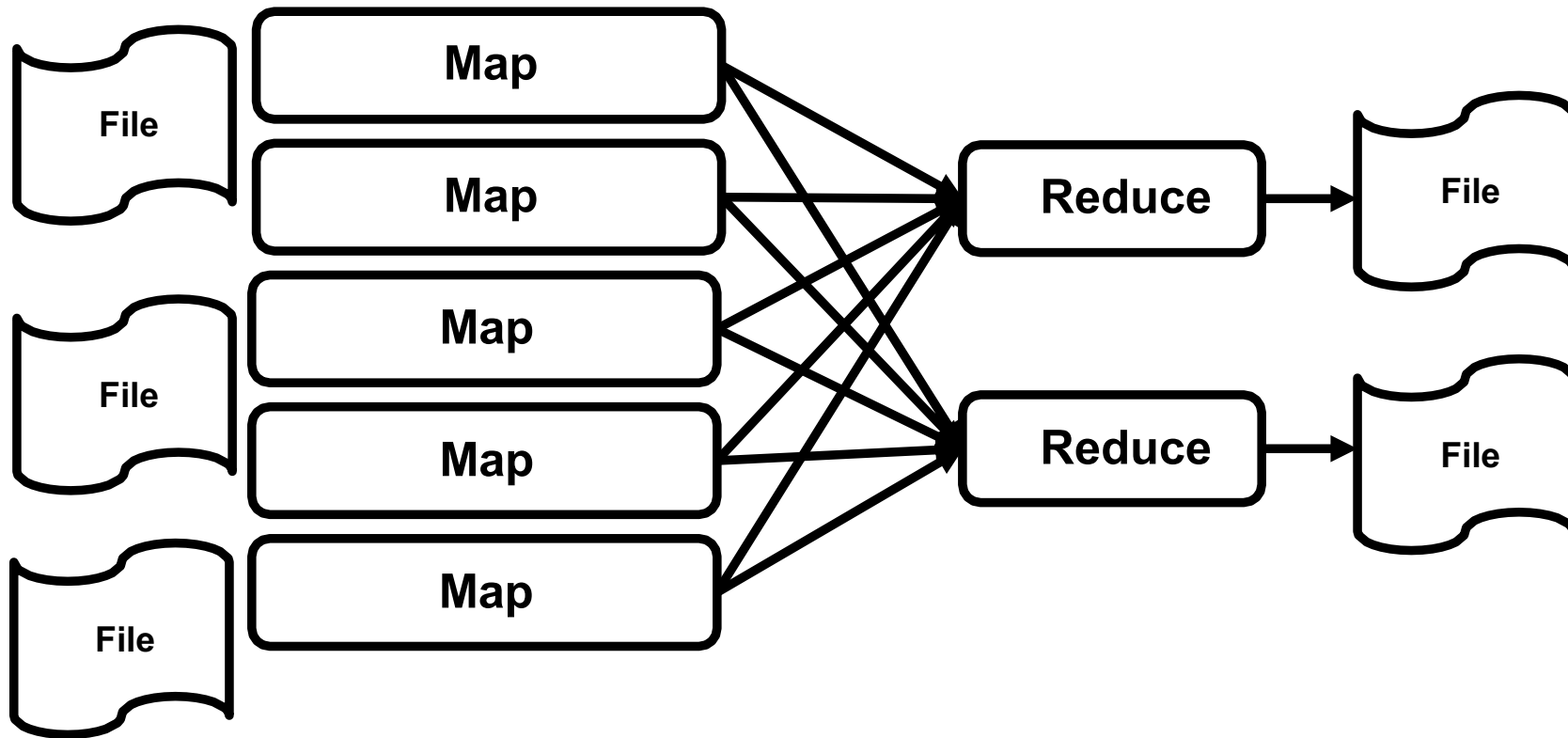
- Reduce function:

$$(K_{inter}, \text{list}(V_{inter})) \rightarrow \text{list}(K_{out}, V_{out})$$

Example: word count

- How can we count how many times each word occurs in a large dataset using only map and reduce?
- Four steps:
 1. Convert files into pairs of (key, value)
 2. Define a map function. Apply to all files.
 3. Shuffle! All elements with same key go to same reduce.
 4. Define a reduce function. Apply to result of the map function.

1000 ft view of Map Reduce



Word count Map Reduce

0.txt
hello world

1.txt
hello edward

2.txt
hello rahul
hello

WC. step 1: to (Key, Value)

0.txt
hello world

→ ("0.txt", "hello world")

1.txt
hello edward

→ ("1.txt", "hello edward")

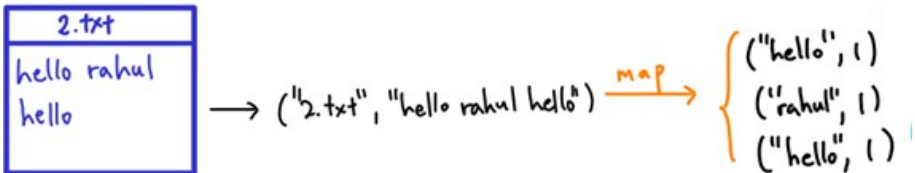
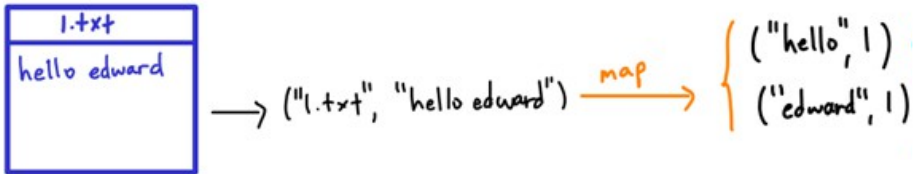
2.txt
hello rahul hello

→ ("2.txt", "hello rahul hello")

Transform file into:
(File Name, List of words)

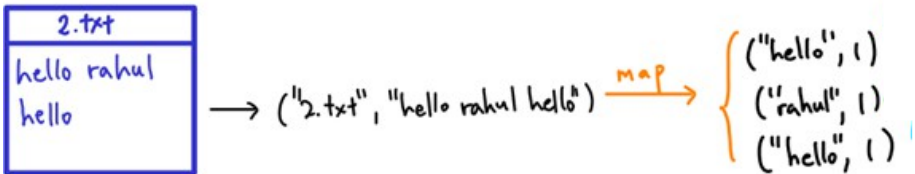
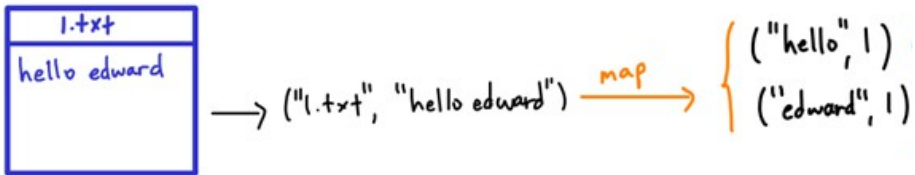
Map function takes (Key, List) and
maps it to List (Key, Value).

WC. step 2: Map Function



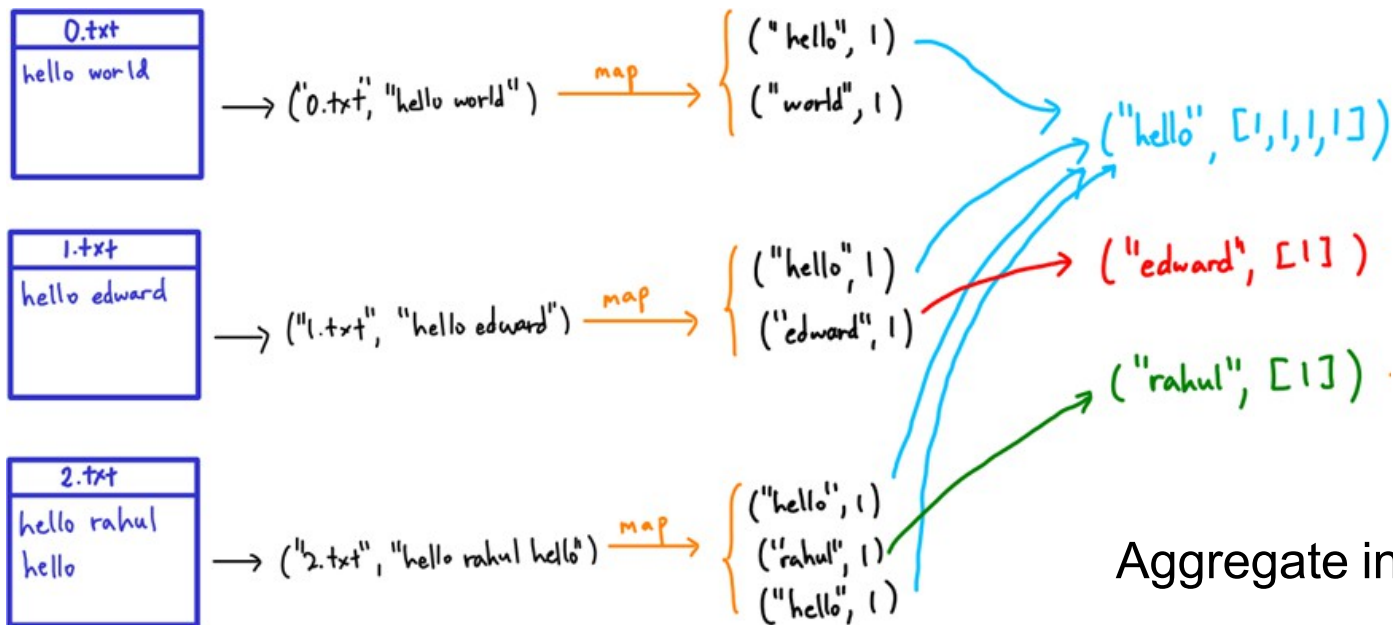
Map function:
Associate each word with a count!

WC. step 2: Map Function



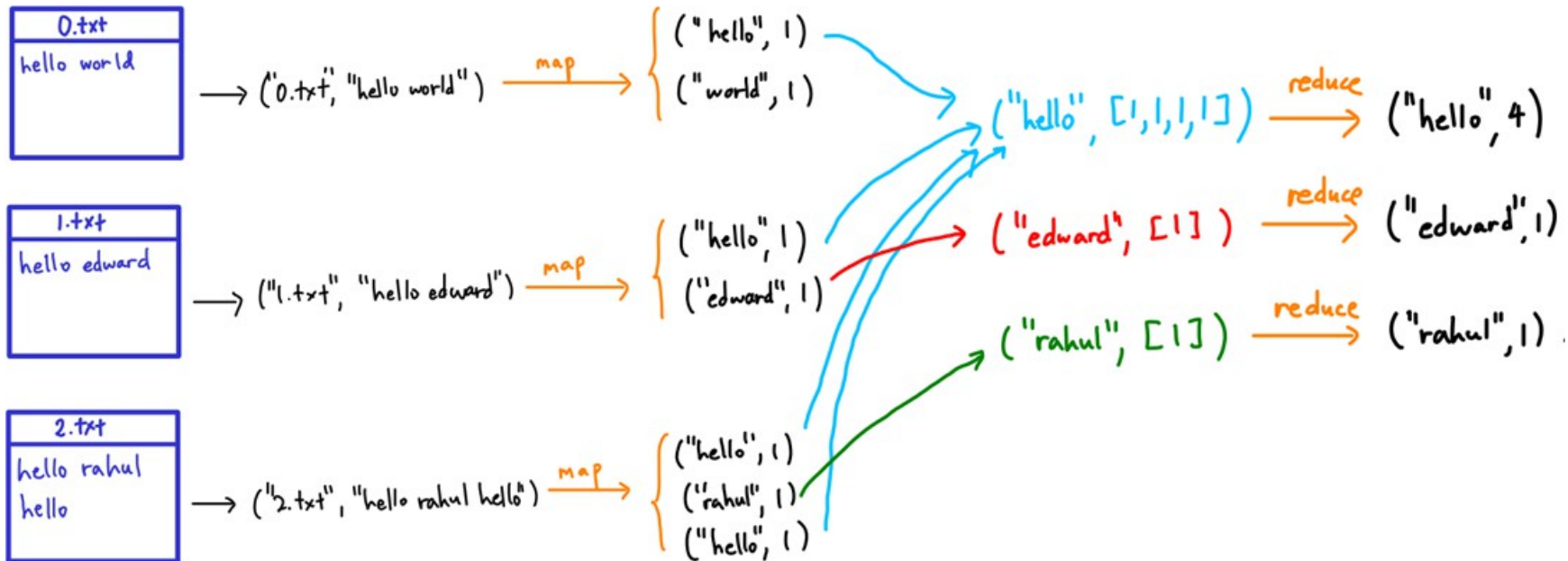
```
map(String key, String value):  
    // key: document name  
    // value: document contents  
    for each word w in value:  
        EmitIntermediate(w, "1");
```

WC. step 3: shuffle!



Aggregate intermediate results by key

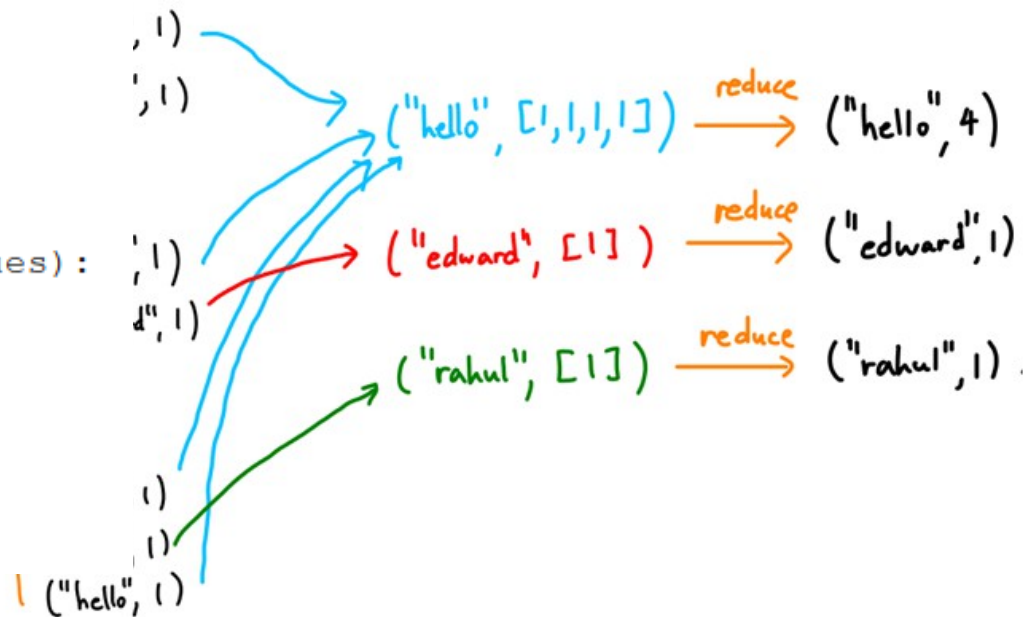
WC. step 4: reduce



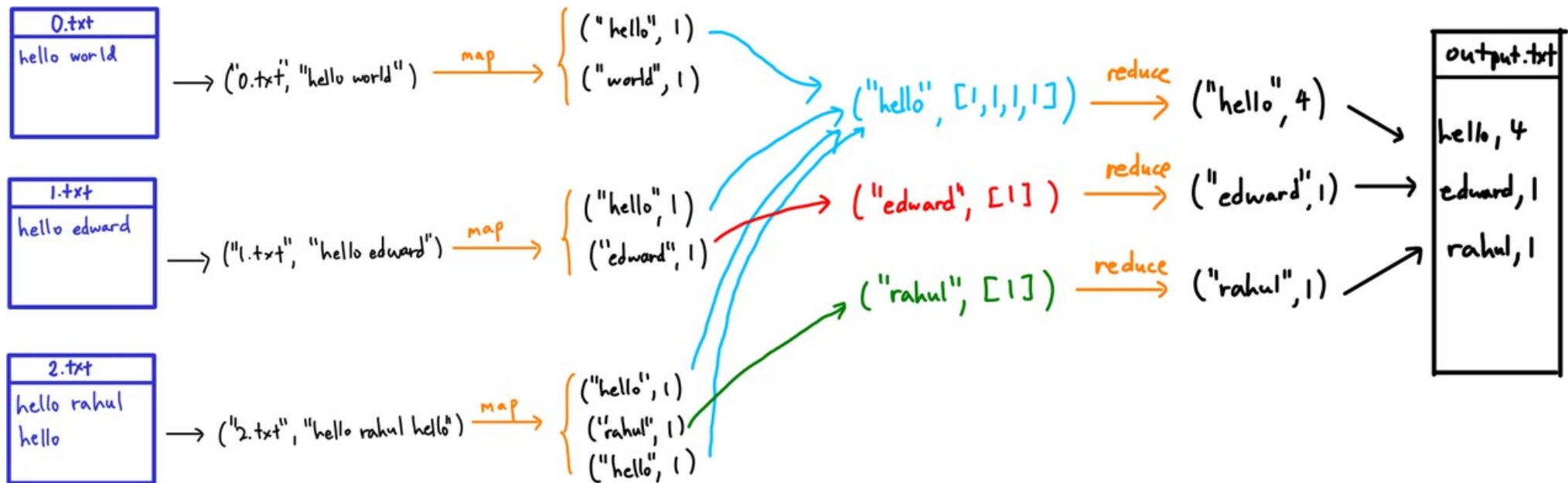
WC. step 4: reduce

```
map(String key, String value):  
  // key: document name  
  // value: document contents  
  for each word w in value:  
    EmitIntermediate(w, "1");
```

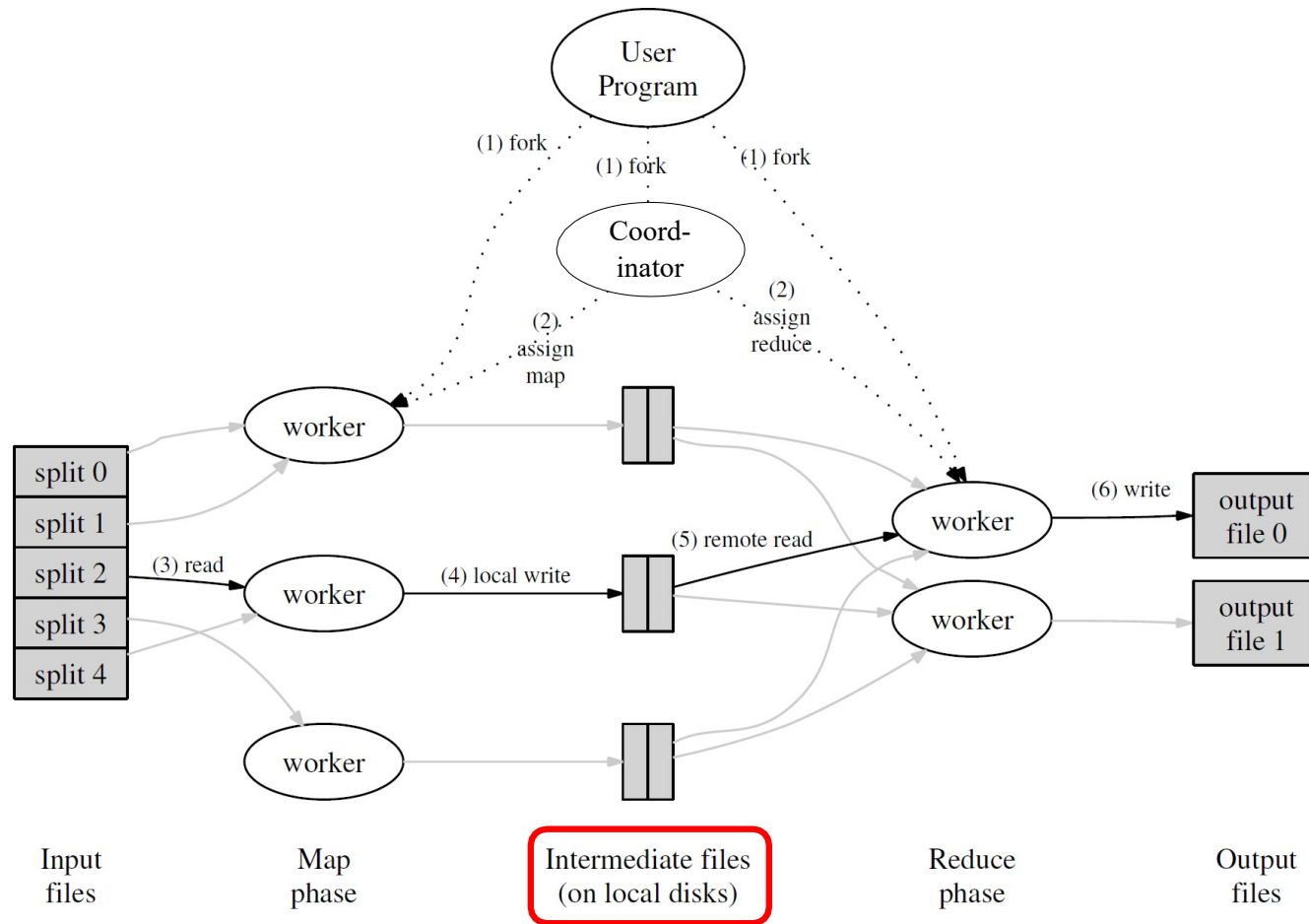
```
reduce(String key, Iterator values):  
  // key: a word  
  // values: a list of counts  
  int result = 0;  
  for each v in values:  
    result += ParseInt(v);  
  Emit(AsString(result));
```



WC. final step, generate output



Map Reduce system components



Fault tolerance

- MapReduce assumes that:
 - Any *individual* machine is unlikely to crash
 - But large cluster of machines is likely to experience failures
- MapReduce does not attempt to handle coordinator crashes
- MapReduce does try to handle worker failures

Fault tolerance in MapReduce

1. If a map or reduce task crashes:

- Retry on another node
 - OK for a map because it had no dependencies
 - OK for reduce because map outputs are on disk
- If the same task repeatedly fails, fail the job or ignore that input block

- Note: for fault tolerance to work, **user tasks must be idempotent**
 - Deterministic and side-effect-free (e.g. not based on time, randomness, etc)

Fault tolerance in MapReduce

2. If a worker node crashes:

- Relaunch its current tasks on other nodes
- Relaunch any maps the node previously ran
 - Necessary because their output files were lost along with the crashed node

Fault tolerance in MapReduce

3. If a task is going slowly (**straggler**):
 - Launch second copy of task on another node
 - Take the output of whichever copy finishes first, and kill the other one
- Critical for performance in large clusters (many possible causes of stragglers)
 - And again requires tasks to be idempotent!

Beyond MapReduce

- Not all applications can easily be expressed with maps & reduces
- MapReduce stores all intermediate state on disk, which is slow
- A lot of other distributed programming frameworks:
 - in-memory processing (Spark),
 - distributed SQL (Apache Hive),
 - graph (PowerGraph),
 - incremental processing (Naiad),
 - streaming (Flink),
 - and many others

Example Apache Spark:

- Open source engine that generalizes the MapReduce model with
 - Higher-level operators: not just map & reduce, but join, filter, SQL, streaming operators, and many built-in libraries
 - Efficient handling of intermediate datasets (e.g. reliable storage in memory)

Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing

Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma,
Murphy McCauley, Michael J. Franklin, Scott Shenker, Ion Stoica
University of California, Berkeley

Abstract

We present Resilient Distributed Datasets (RDDs), a distributed memory abstraction that lets programmers perform in-memory computations on large clusters in a fault-tolerant manner. RDDs are motivated by two types of applications that current computing frameworks handle inefficiently: iterative algorithms and interactive data mining tools. In both cases, keeping data in memory can improve performance by an order of magnitude. To achieve fault tolerance efficiently, RDDs provide a restricted form of shared memory, based on coarse-grained transformations rather than fine-grained updates

tion, which can dominate application execution times.

Recognizing this problem, researchers have developed specialized frameworks for some applications that require data reuse. For example, Pregel [22] is a system for iterative graph computations that keeps intermediate data in memory, while HaLoop [7] offers an iterative MapReduce interface. However, these frameworks only support specific computation patterns (e.g., looping a series of MapReduce steps), and perform data sharing implicitly for these patterns. They do not provide abstractions for more general reuse, e.g., to let a user load several datasets into memory and run ad-hoc queries across them.



Key idea in Spark

- Resilient Distributed Datasets (RDDs)
 - Immutable collections of objects that can be stored in memory or disk across a cluster
 - Built with parallel transformation operators (map, filter, ...)
 - Automatically rebuilt on failure
- Think of these as a form of distributed, reliable virtual memory!

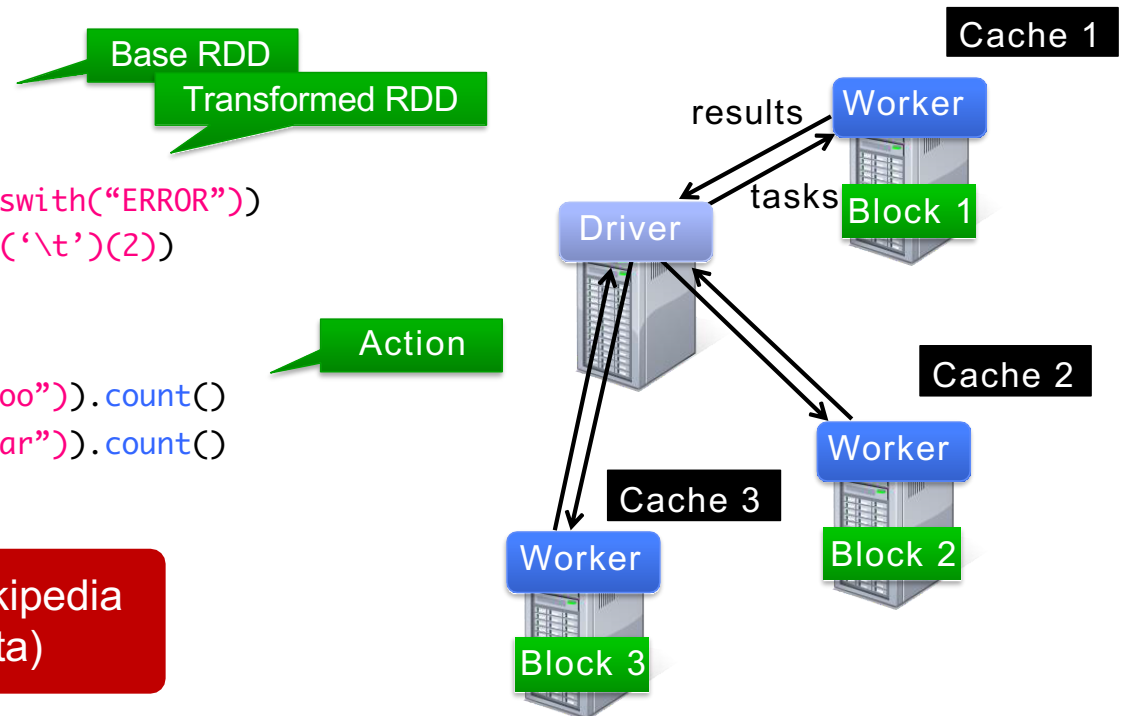
Example: log mining

- Load error messages from a log into memory, then interactively search for various patterns

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split('\t')(2))
messages.cache()
```

```
messages.filter(lambda s: s.contains("foo")).count()
messages.filter(lambda s: s.contains("bar")).count()
. . .
```

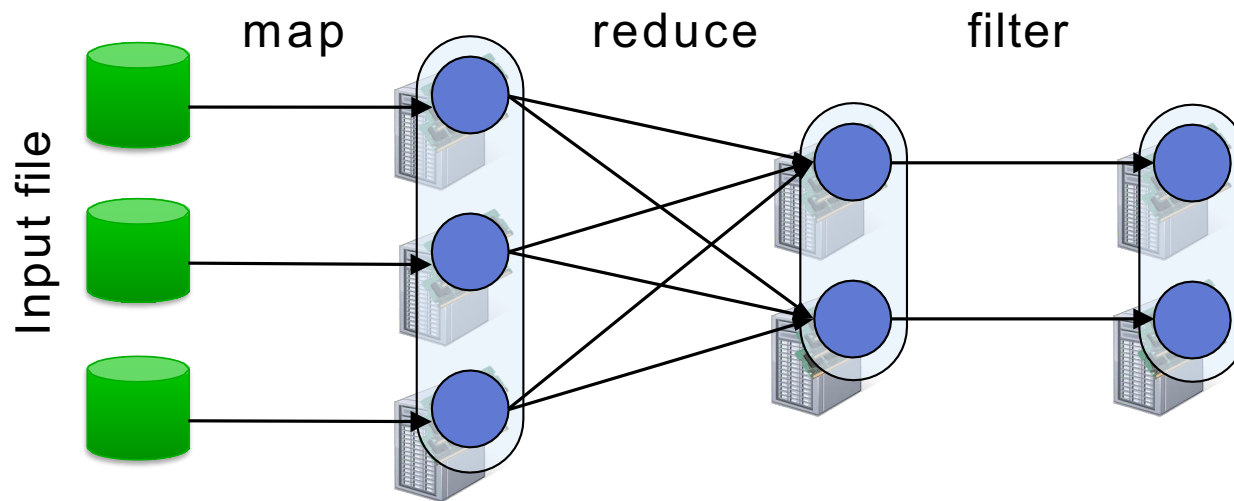
Performance: full-text search of Wikipedia
in 1 sec (vs 40 s for on-disk data)



RDDs: fault tolerance

- RDDs track *lineage* info to rebuild lost data

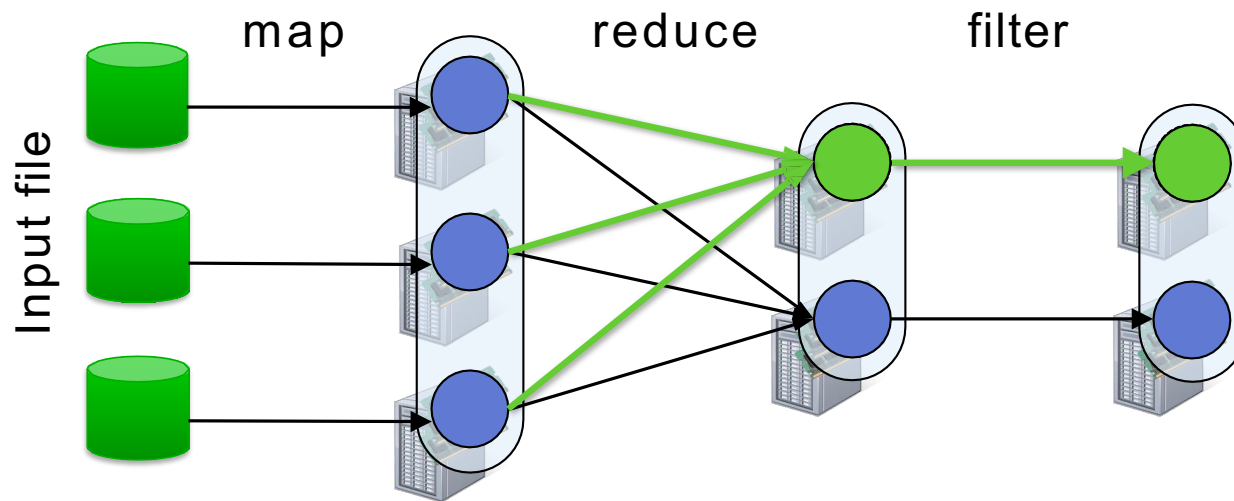
```
file.map(record => (record.type, 1))  
  .reduceByKey((x, y) => x + y)  
  .filter((type, count) => count > 10)
```



RDDs: fault tolerance

- RDDs track *lineage* info to rebuild lost data

```
file.map(record => (record.type, 1))  
    .reduceByKey((x, y) => x + y)  
    .filter((type, count) => count > 10)
```



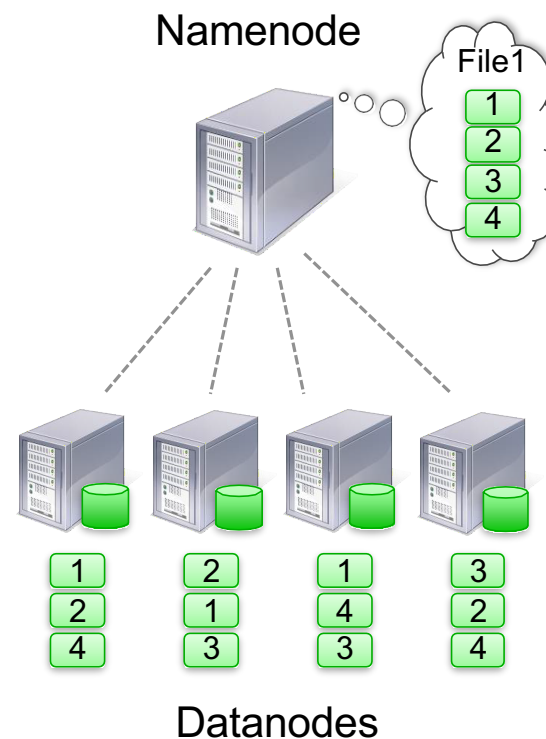
Hadoop components

- Distributed file system (HDFS)
 - Single namespace for entire cluster
 - Replicates data 3x for fault-tolerance
- MapReduce framework
 - Runs jobs submitted by users
 - Manages work distribution & fault-tolerance
 - Colocated with file system



Hadoop Distributed File System

- Files split into 128MB blocks
- Blocks replicated across several datanodes (often 3)
- Namenode stores metadata (file names, locations, etc)
- Optimized for large files, sequential reads
- Files are append-only



Coordination

Topic roadmap

Distributed File Systems

Peer-To-Peer System:
The Internet

Distributed Data Processing

Coordination
(Atomic Commit and Consensus)

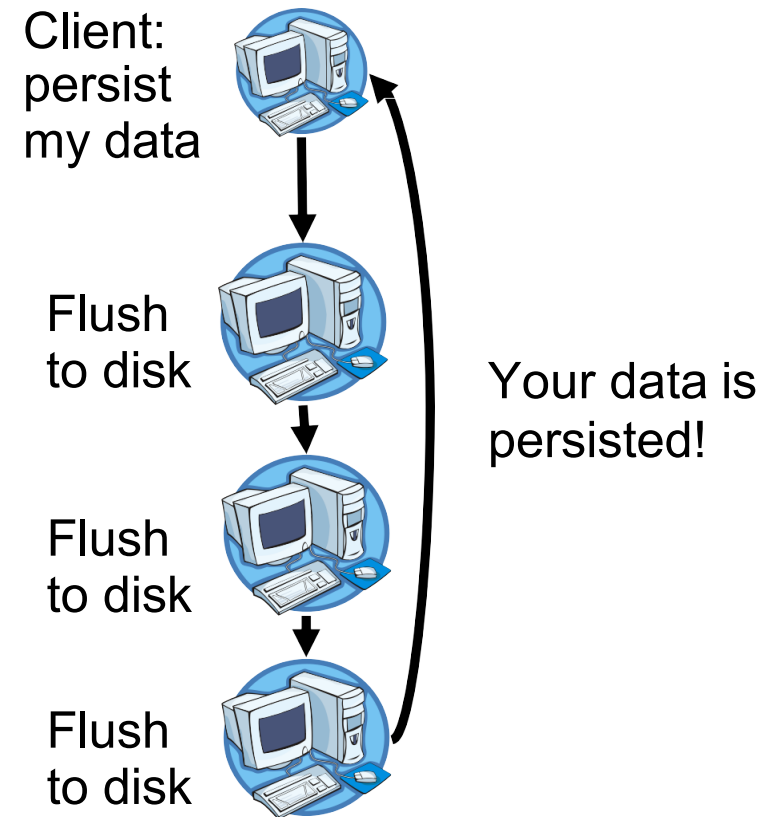
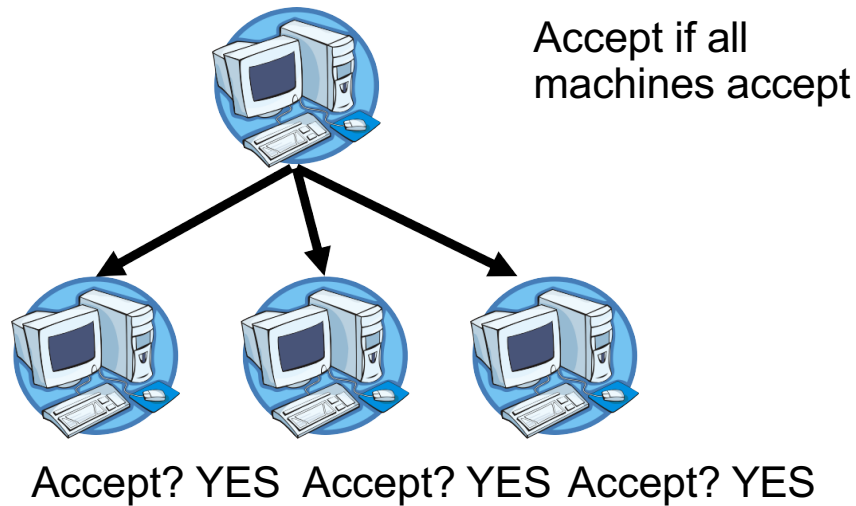
Coordination: making distributed decisions

- Functionality is spread across machines. Requires coordination to reach distributed decision
- Distributed protocols are hard!

Coordination is hard!

- When machines can fail!
- When networks are slow and/or unreliable
- When machines may receive conflicting proposals on what to do

Coordination: making distributed decisions



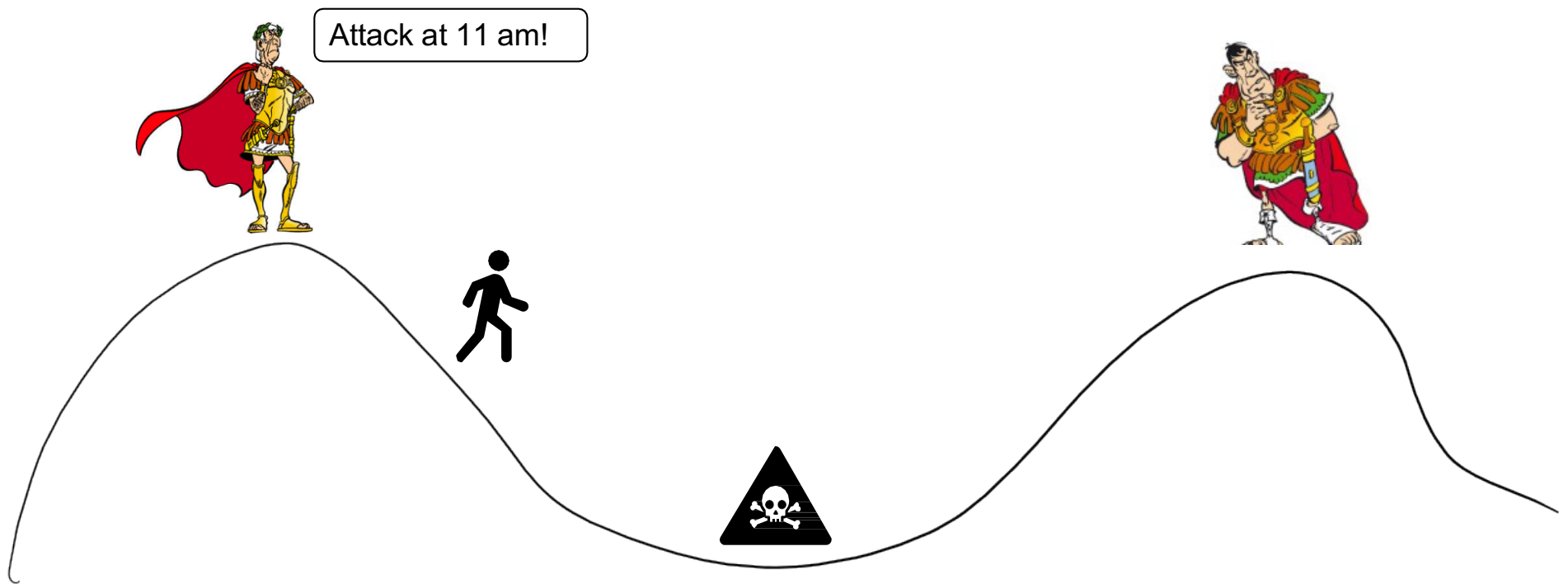
Agreeing simultaneously: General's Paradox

- Two generals, on separate mountains
 - Can only communicate via messengers
 - Messengers can be captured

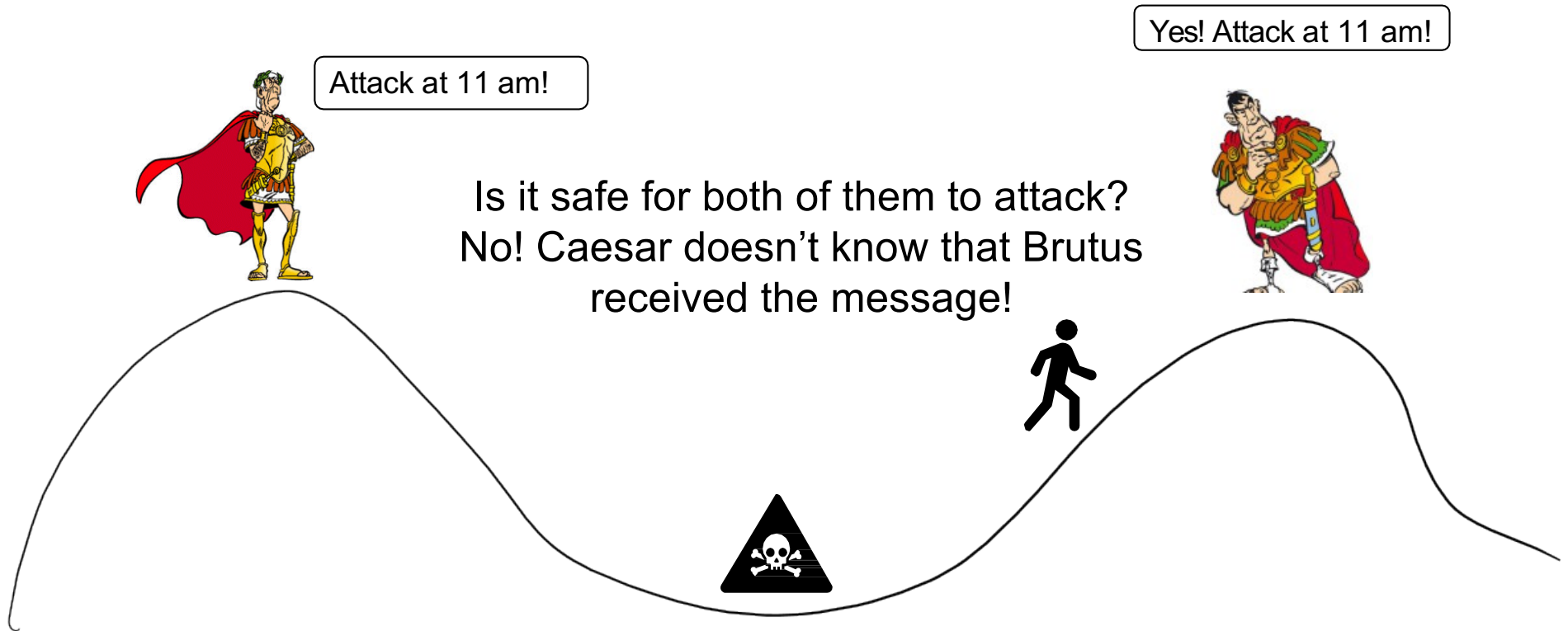


- Problem: need to coordinate attack
 - If they attack at different times, they all die
 - If they attack at same time, they win

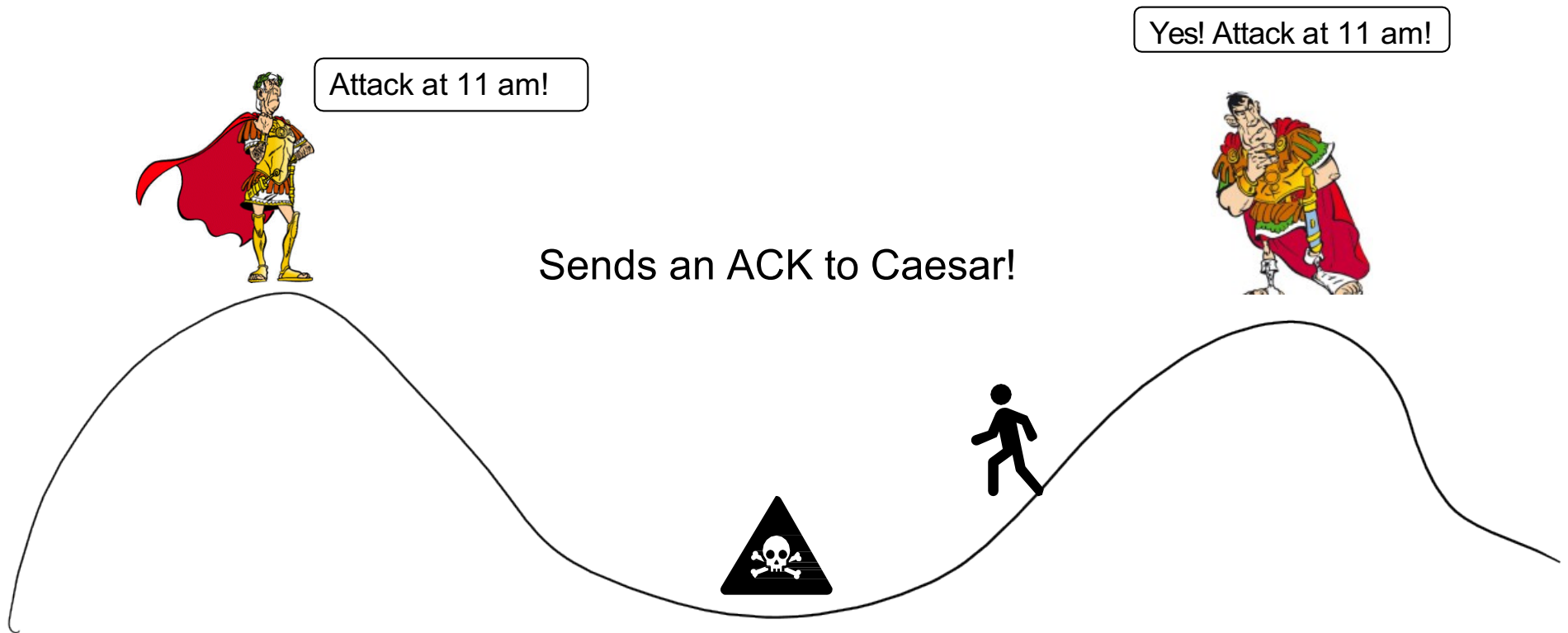
General's Paradox: scenario 1



General's Paradox: scenario 1



General's Paradox: scenario 1



General's Paradox: scenario 1

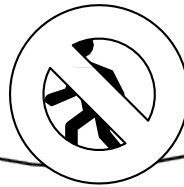


Attack at 11 am!



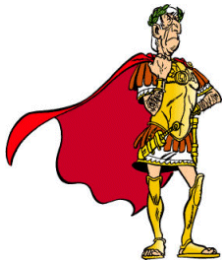
Yes! Attack at 11 am!

Now is it safe?
No! Messenger could have been attacked



General's Paradox: scenario 1

Caesar needs to know that
Brutus knows that
Caesar knows that
Brutus knows that
They are attacking at 11 am



Impossible to achieve
simultaneous actions with
unreliable channels because
never know whether messenger
or ACK got lost

Agreeing simultaneously: General's Paradox

- If the network is unreliable, it is impossible to guarantee two entities do something simultaneously
- If nodes behave maliciously, impossible to get eventual agreement if there are less than $3f+1$ parties present (of which f can misbehave)
- Entire textbook on impossibility results in distributed computing
...

Practical alternative: agree eventually

- Two or more machines agree on something eventually
- No constraints on time, just that it will eventually happen!
- Examples:
 - 2-Phase Commit: agree to commit a transaction (atomic commit)
 - Paxos: agree on a value or sequence of values (consensus)

Two-Phase Commit (2PC)

- Developed by Turing award winner Jim Gray
 - (first Berkeley CS PhD, 1969)
- Many important database breakthroughs also from Jim Gray



Jim Gray

Two-Phase Commit (2PC)

- **Goal: determine whether system should commit or abort a transaction**
- All processes that reach a decision reach the same one (**Agreement**)
- A process cannot reverse its decision after it has reached one (**Finality**)
- If there are no failures and every process votes yes, the decision will be commit (**Consistency**)
- If all failures are repaired and there are no more failures, then all processes will eventually decide commit/abort (**Termination**)

2PC terminology

- Setup:
 - One coordinator
 - A set of participants
- Each process has access to a persistent log:
 - Processes can crash and recover.
 - Recorded information on the log will persist after crashes

2PC terminology

- Coordinator asks all processes to vote
- Each participant (including coordinator) can vote either YES or NO
 - If all vote YES, coordinator must vote COMMIT
 - If one of them votes NO, coordinator must vote ABORT

2PC: The easy case (no failures)

Coordinator Algorithm

1. Coordinator sends **VOTE-REQ** to all workers

3. Collect votes

- If receive **VOTE-COMMIT** from all N workers, send **GLOBAL-COMMIT** to all workers
- If don't receive **VOTE-COMMIT** from all N workers, send **GLOBAL-ABORT** to all workers

Worker Algorithm

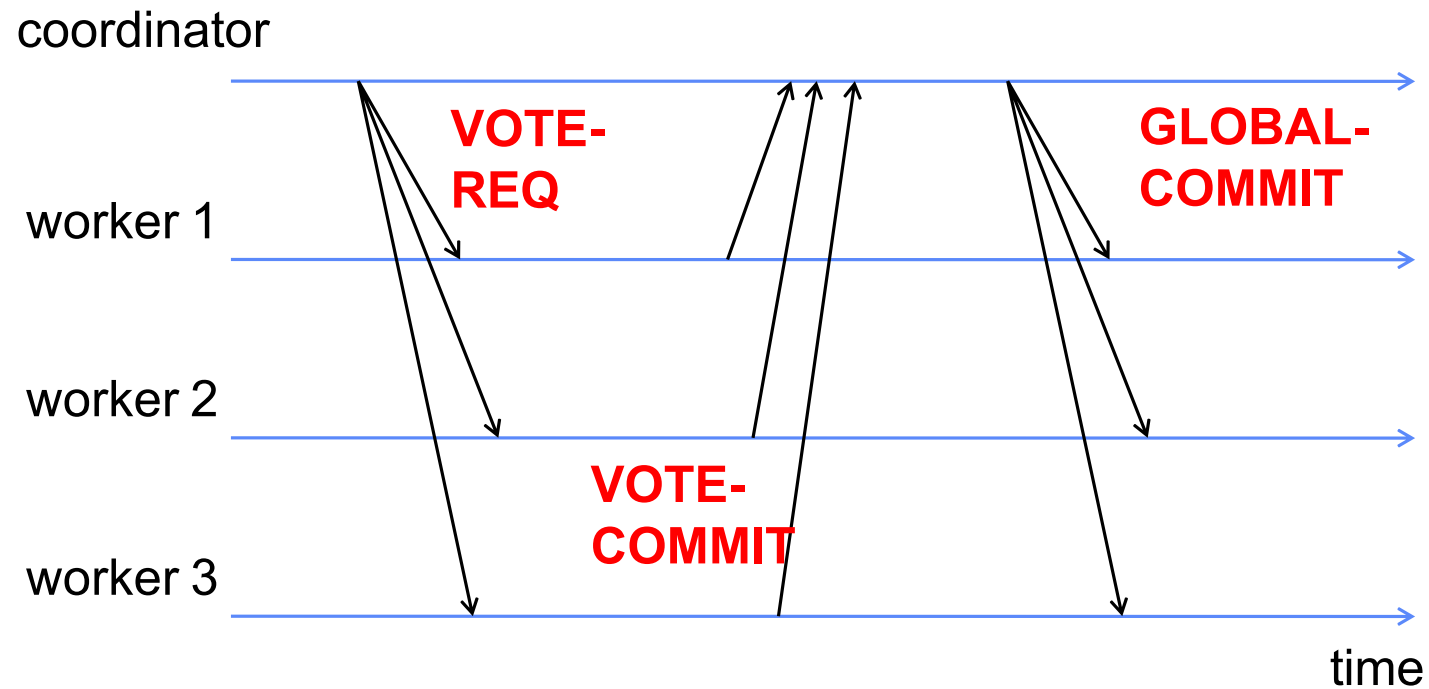
2.

- Send **VOTE-COMMIT** or **VOTE-ABORT** to coordinator
- If sent **VOTE-ABORT** immediately abort

4.

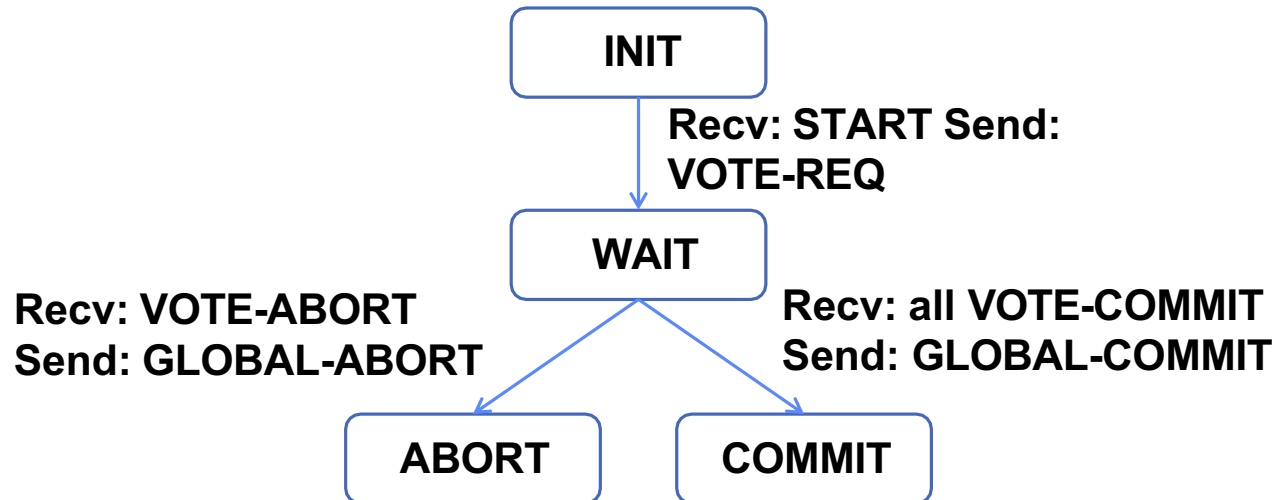
- If receive **GLOBAL-COMMIT** then commit
- If receive **GLOBAL-ABORT** then abort

Failure free example execution

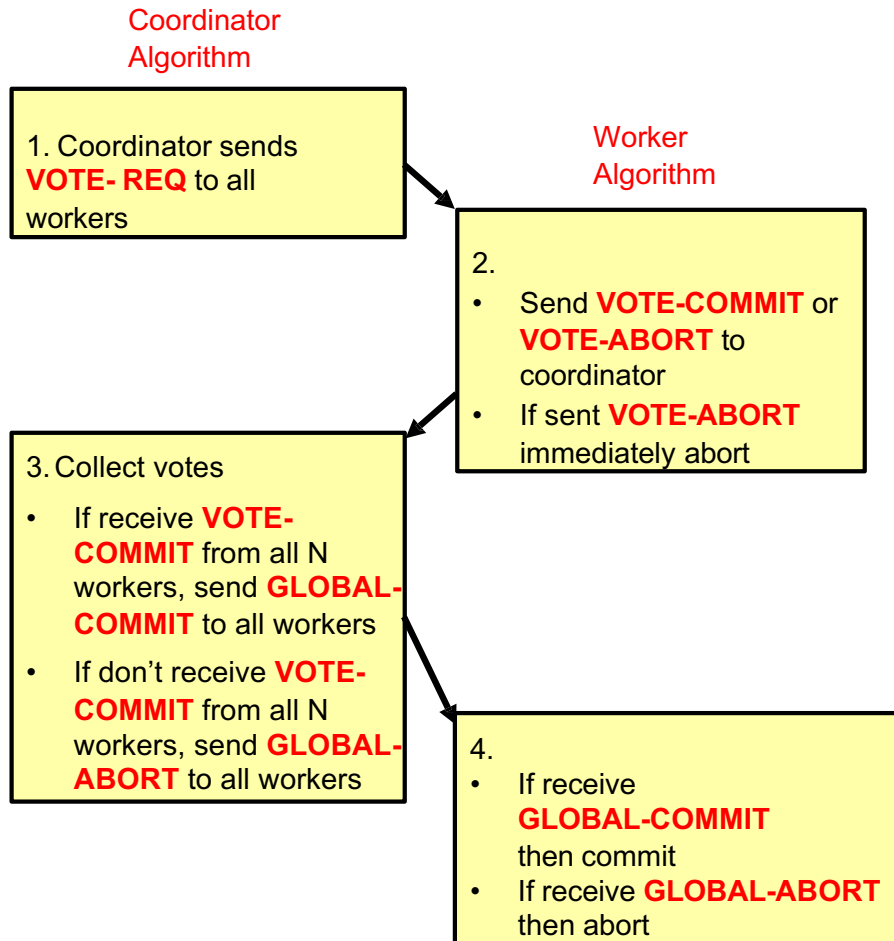


State machine of coordinator

Coordinator implements
simple state machine



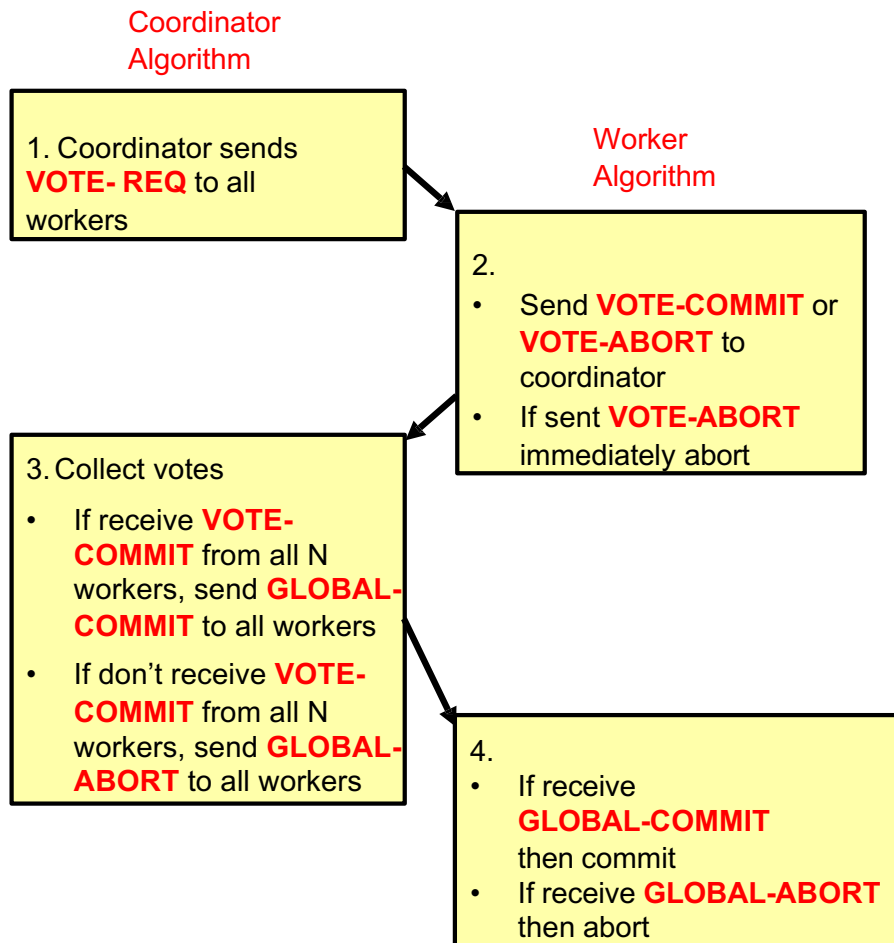
What about failures?



1) What happens when waiting for a message that never comes?

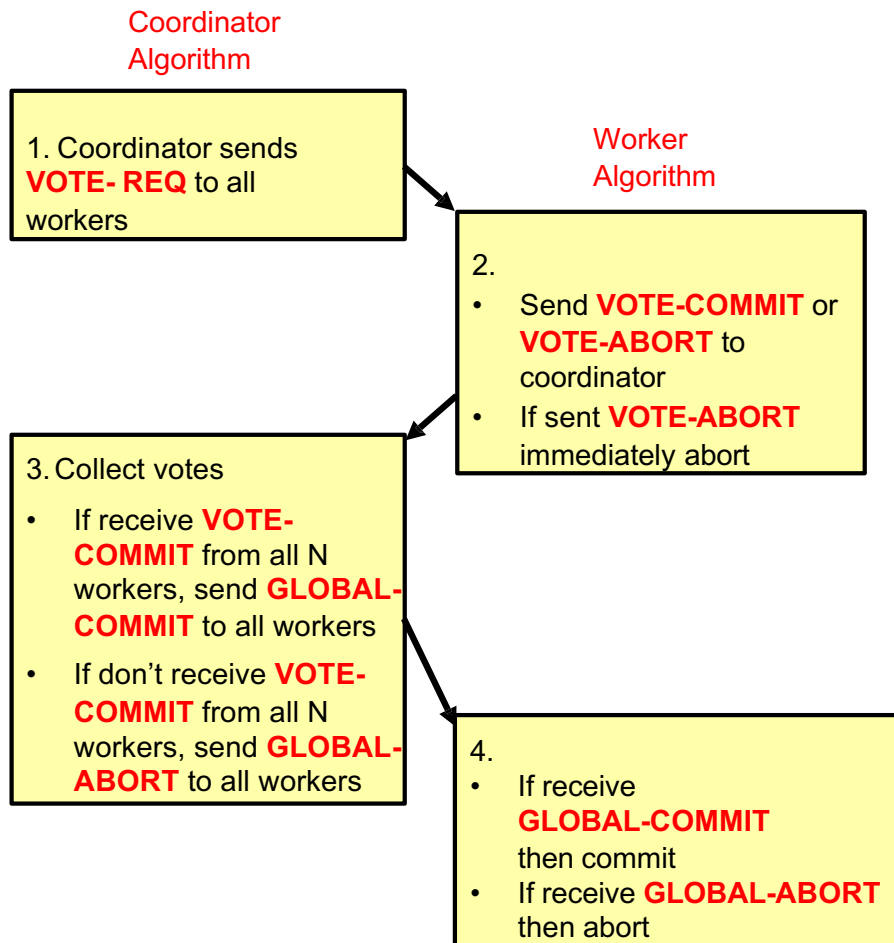
2) What happens during when participant recovers from a failure?

What happens when a message never comes?



- Step 2: worker waiting from VOTE-REQ from coordinator
- Step 3: Coordinator is waiting for vote from participants
- Step 4: Worker who voted YES is waiting for decision

What happens when a message never comes?

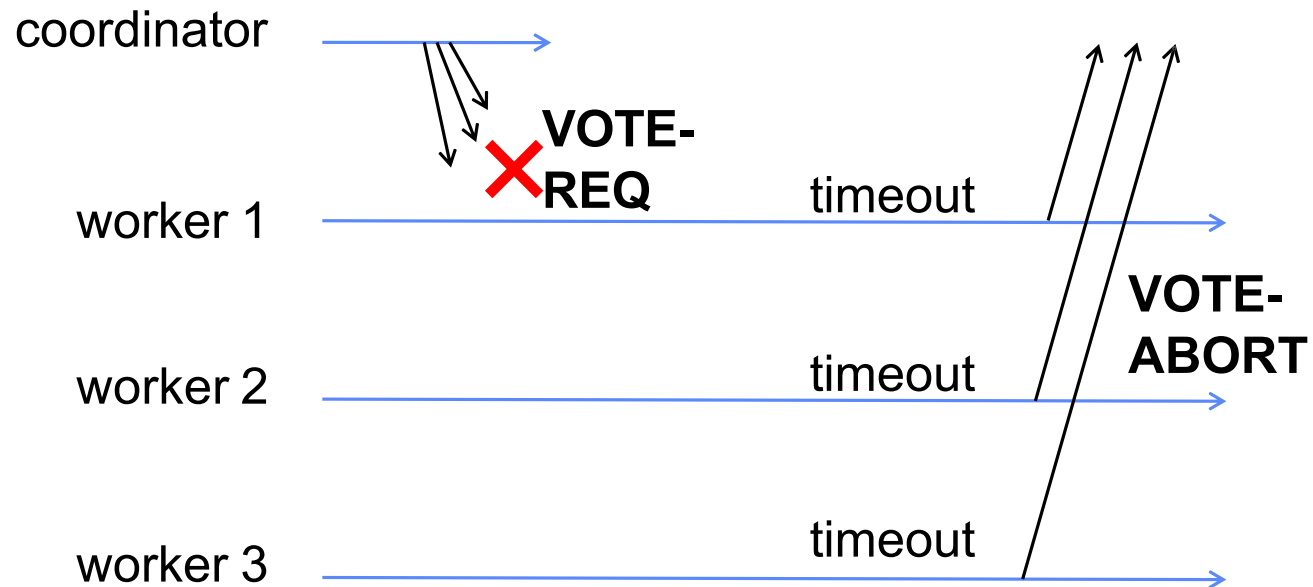


- Step 2: worker waiting from VOTE-REQ from coordinator
 - Since it has not cast its vote yet, worker can decide abort and halt
- Step 3: Coordinator is waiting for vote from participants
 - Coordinator can always vote abort itself, so votes abort and sends **GLOBAL-ABORT** to all participants
- Step 4: Worker who voted YES is waiting for decision
 - Worker cannot decide: it must run a termination protocol

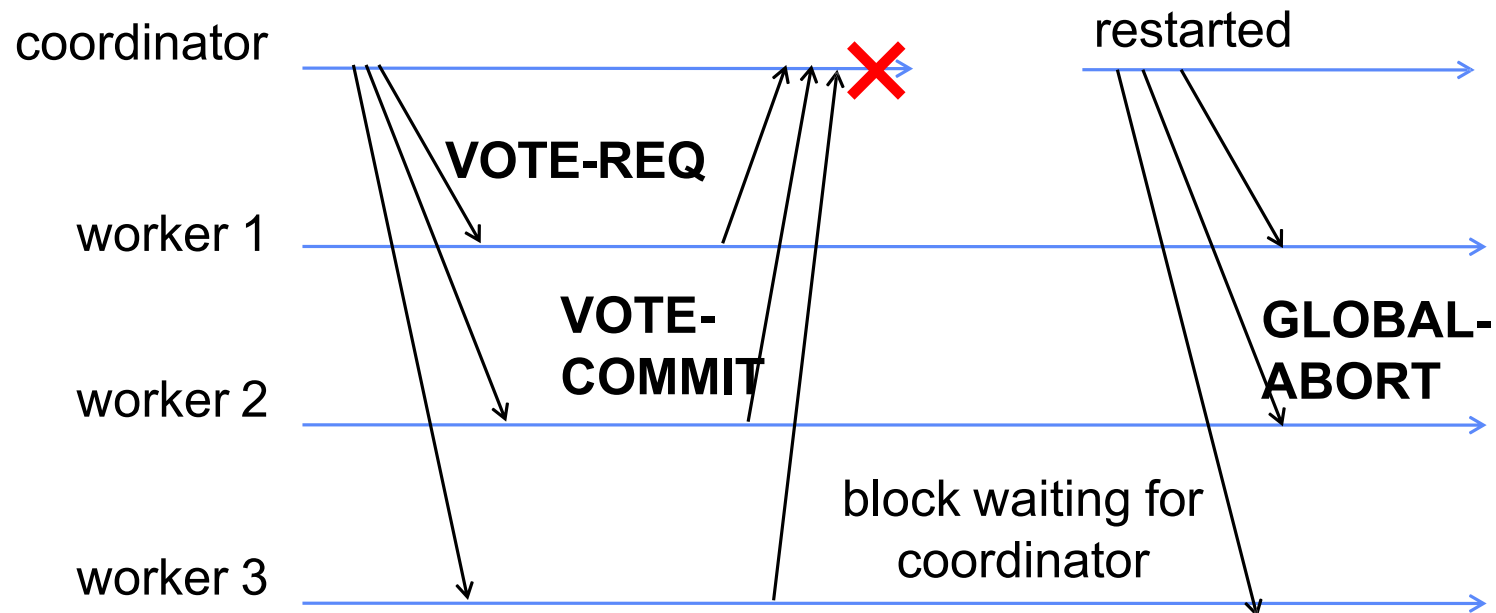
Termination protocol

- Option 1: Simply wait for coordinator to recover.
 - *If all failures are repaired and there are no more failures, then all processes will eventually decide commit/abort (Termination)*
=> No need to recover until coordinator has recovered
- (Better) Option 2: Ask a friendly participant p
 - Case 1: *If p has decided COMMIT/ABORT, forwards decision to initiator*
 - Case 2: *If P has not decided, votes ABORT, sends abort to initiator. Initiator knows decision will be ABORT. So can decide*
 - Case 3: *If P has voted COMMIT, P is also stuck and can't help initiator*
- If every participant voted COMMIT and coordinator crashes before sending decision, must wait for coordinator to recover to decide!

Example of coordinator failure #1



Example of coordinator failure #2



2PC summary

- Why is 2PC not subject to the General's paradox?
 - Because 2PC is about all nodes eventually coming to the same decision – not necessarily at the same time!
 - Allowing us to reboot and continue allows time for collecting and collating decisions
- Biggest downside of 2PC: blocking
 - A failed node can prevent the system from making progress
 - Still one of the most popular coordination algorithms today

Alternatives to 2PC

- **Three-Phase Commit:** One more phase, does not have to block if one node fails.
- **Paxos:** An alternative that does not have the 2PC blocking problem
 - Developed by Leslie Lamport (Turing Award Winner)
 - No fixed leader, can choose new leader on fly, deal with failure
 - Often extended to agree on a sequence of values (log)
- What happens if one or more of the nodes is malicious?
 - **Malicious:** attempting to compromise the decision making
 - Use a more hardened decision-making process:
Byzantine Agreement and **Blockchains**

A glance at Paxos paper

- Published at ACM Transactions on Computer Systems (TOCS), 1998

The Part-Time Parliament

LESLIE LAMPORT

Digital Equipment Corporation

Recent archaeological discoveries on the island of Paxos reveal that the parliament functioned despite the peripatetic propensity of its part-time legislators. The legislators maintained consistent copies of the parliamentary record, despite their frequent forays from the chamber and the forgetfulness of their messengers. The Paxos parliament's protocol provides a new way of implementing the state-machine approach to the design of distributed systems.

Thanks